Sync_car

Objetivo

Desenvolver um sistema de sincronização de threads para automatizar e ordenar o embarque simultâneo de passageiros nos vagões da METROREC. A solução deve gerenciar a concorrência de forma segura, utilizando estritamente um único mutex associado a variáveis de condição, garantindo a total ausência de semáforos e de espera ativa (busy-wait).

Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
`sync_car.c` com `struct station`, `station_init` e as três funções	OK	`make && ./sync_car` compila e roda sem avisos no seu código
`station_load_car` só retorna com o carro cheio ou todos os que esperavam embarcados	OK	Teste do professor (`Looks good!`) e 500 rodadas em loop
`station_wait_for_car` só retorna com carro na estação e assento livre	OK	Teste do professor: número liberado por carro igual a `min(esperando, assentos)`
`station_on_board` avisa o carro que o passageiro embarcou	OK	o carro depende desse aviso para retornar
Nenhum passageiro depende do `on_board` de outro para ser liberado	OK	os cenários do `teste_bordas` com `on_board` lento passam
Apenas as funções `pthread` listadas	OK	grep -n "pthread_" sync_car.c
Sem semáforos nem outras primitivas	OK	`grep -n "sem_" sync_car.c` não retorna nada
Um único lock por `struct station`	OK	A struct tem só `pthread_mutex_t lock`
Um carro por vez na estação	OK	Premissa do enunciado, que o código assume
Sem busy-wait	OK	Toda espera é `pthread_cond_wait` dentro de `while`
Makefile gera o executável `sync_car`	OK	make clean && make

Executável em Linux, Unix ou macOS	OK	Testado em Linux (WSL). Código usa apenas pthread_mutex e pthread_cond, portáveis para Unix e macOS
README	OK	Presente dentro do repositorio no github

Como Reproduzir

Compilar:

```
make clean
make
```

Executar:

```
./sync_car
```

Arquitetura (resumida)

Arquivos e responsabilidades:

1. sync_car.h → define a `struct station` (um mutex, duas variáveis de condição e três contadores) e os protótipos das quatro funções.

2. sync_car.c → implementa a logica central do codigo, `station_init`, `station_load_car`, `station_wait_for_car` e `station_on_board`.

3. `sync_car-runner.c` → teste fornecido pelo professor, cria as threads de passageiros e de carro.

Decisões técnicas:

1. **Reserva antecipada de assentos → A decrementação de assentos e o incremento de `passageiros_embarcando` foram posicionados dentro de `station_wait_for_car` (sob o *lock*) → garante que o limite de lotação seja respeitado atomicamente antes do retorno das *threads* de passageiros.**

2. **Uso de duas variáveis de condição → A separação entre `carro_chegou` e `espera_embarque` isola os fluxos de espera de passageiros e do vagão → O vagão utiliza `broadcast` para acordar múltiplos passageiros simultaneamente, enquanto o embarque utiliza `signal` para notificar individualmente o carro.**

3. **Condição de partida do vagão → O vagão só prossegue quando todos os assentos possíveis são preenchidos ou a fila se esgota → A inclusão da verificação de `passageiros_embarcando > 0` no laço evitou que o vagão partisse prematuramente antes da conclusão física dos**

embarques.

4. **Notificação otimizada em `station_on_board` → O envio de sinal ao carro foi restrito ao momento em que o contador de `passageiros_embarcando` atinge zero → reduz trocas de contexto desnecessárias, garantindo que o vagão só desperte quando o ciclo de lotação puder ser reavaliado.**

Estratégias e Diário de Desenvolvimento

Estratégias da semana

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Mutex único + 2 condições + 3 contadores	Um mutex, `carro_chegou` (passageiros dormem) e `espera_embarque` (carro dorme), contadores de esperando, assentos livres e embarcando	Não houve troca

Diário de Tentativas

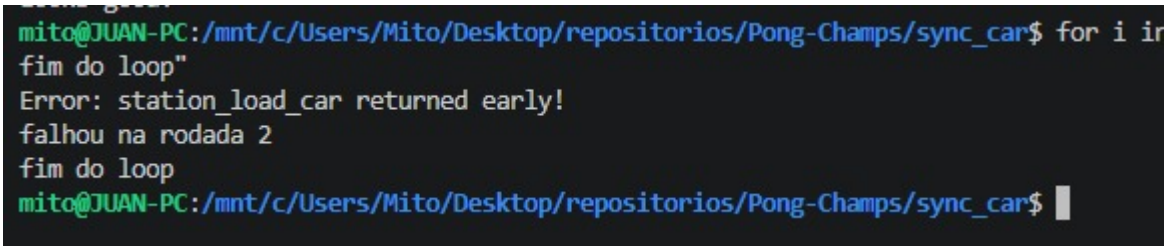
#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Defini a `struct station` (1 mutex, 2 condições, 3 contadores)	OK	-	01/11	commit `b4f8032`
2	S1	Escrevi `station_init`, `station_on_board` e `station_wait_for_car`	OK	-	01/11	commits `8253bdd`, `1c9e4e8`, `92034ec`
3	S1	Escrevi `station_load_car` com o `while` só em `(livres > 0 && esperando > 0)`	OK	-	01/11	commit `3be8f46`
4	S1	Criei o `sync_car.h` e o Makefile	OK	-	02/11	commits `8ad9ebe`, `f9fa121`
5	S1	Rodei o arquivo de teste em loop	Falha / Error: station_load_car returned early!	O carro acorda quando `embarcando` chega a 0 de forma transitória(em	02/11	linhas 121–129 (erros nas linhas 123 e 127) do

				um curto espaço de tempo), sem checar `embarcando > 0`, saindo com liberados que ainda não sentaram		evidencias.log
6	S1	Acrescentei `||` `passageiros_embarcando > 0` ao `while` do carro e repeti o loop (100 e depois 500 rodadas)	OK	-	02/11	commit `740ee7a` / 137 e 138 (100 rodadas) / 163 e 164 (500 rodadas) do evidencias.log
7	S1	Rodei com `-fsanitize=thread`	Parcial	Avisos só em `sync_car-runner.c` (variáveis globais e threads sem `join`)	02/11	Linhas 165 - 257 evidencias.log
8	S1	Rodei o `teste_bordas` (arquivo de testes extras criado pelo claude)	OK	-	02/11	Linhas 263 - 363 evidencias.log
9	S1	rodei o `teste_invalidos.c` (carro com assentos negativos: -1, -1000 e INT_MIN)	OK	-	04/11	Linhas 1275 - 1315 evidencias.log

Também rodei o teste_bordas com alterações temporárias no código (por exemplo, trocando while por if em station_wait_for_car), só para confirmar que o teste acusa falhas e não dá falso positivo. Essas execuções aparecem no evidencias.log (582 - 695) com resultado FALHOU, e o código foi restaurado em seguida.

Prints

- **Error: station_load_car returned early!**
  - O carro é acordado quando o contador de passageiros embarcando chega a 0, mas esse 0 pode durar pouco, ou seja, outro passageiro pode reservar um assento antes de o carro voltar a rodar.



- **Validação final: mudei o while e os testes passaram**
  - o carro confere de novo `passageiros_embarcando > 0` ao acordar

```
mito@JUAN-PC:/mnt/c/Users/Mito/Desktop/repositorios/Pong-Champs/sync_car$    ./sync_car
car entering station with 0 free seats
car departed station with 0 new passenger(s) (expected 0)
car entering station with 47 free seats
car departed station with 47 new passenger(s) (expected 47)
car entering station with 7 free seats
car departed station with 7 new passenger(s) (expected 7)
car entering station with 12 free seats
car departed station with 12 new passenger(s) (expected 12)
car entering station with 29 free seats
car departed station with 29 new passenger(s) (expected 29)
car entering station with 24 free seats
car departed station with 5 new passenger(s) (expected 5)
Looks good!
mito@JUAN-PC:/mnt/c/Users/Mito/Desktop/repositorios/Pong-Champs/sync_car$
```

- **Comando make estava gerando warnings**
  - Variável que não era utilizada no arquivo de testes

```
mito@JUAN-PC:/mnt/c/Users/Mito/Desktop/repositorios/Pong-Champs/sync_car$ make
gcc -Wall -Wextra -pthread -o sync_car sync_car-runner.c sync_car.c
sync_car-runner.c: In function 'alarm_handler':
sync_car-runner.c:54:24: warning: unused parameter 'foo' [-Wunused-parameter]
   54 | void alarm_handler(int foo)
      |                    ~~~~^~~
mito@JUAN-PC:/mnt/c/Users/Mito/Desktop/repositorios/Pong-Champs/sync_car$
```

- **o Makefile foi reestruturado para compilar os arquivos de objeto (`.o`) separados**
  - o alerta de "`unused parameter`" foi silenciado exclusivamente no arquivo de testes.

```
mito@JUAN-PC:/mnt/c/Users/Mito/Desktop/repositorios/Pong-Champs/sync_car$ make
gcc -Wall -Wextra -pthread -c sync_car.c
gcc -Wall -Wextra -pthread -Wno-unused-parameter -c sync_car-runner.c
gcc -Wall -Wextra -pthread -o sync_car sync_car.o sync_car-runner.o
mito@JUAN-PC:/mnt/c/Users/Mito/Desktop/repositorios/Pong-Champs/sync_car$
```

Uso de IA

Onde usei IA:

Utilizei IA para organização inicial (criação do fluxo de trabalho e cronograma da semana), compreensão do enunciado (desconstruindo-o em partes menores), entendimento do comportamento das funções da biblioteca `pthreads` exigidas pelo professor, e geração de uma bateria de testes complementares para mapear casos de borda e evitar que detalhes passassem despercebidos.

Prompts principais (liste, sem história):

- Me ajude a criar um cronograma para essa implementação
- Me diga o que cada uma dessas funções (`pthread_mutex_*`, `pthread_cond_*`) faz.
- Qual a diferença do `cond_signal` para o `cond_broadcast`?
- Você consegue fazer uma bateria de testes com casos de borda
- Como eu posso garantir que nao tem busy wait?

O que validei manualmente e como:

A lógica de `sync_car.c` foi escrita por mim. Validei rodando o teste do professor em loops de 100 e 500 rodadas (que revelou o `returned early`), com o ThreadSanitizer (nenhum aviso aponta para `sync_car.c`, os 5 avisos são do arquivo de teste) e com o `teste_bordas`. Conferi os números de linha do log e os commits com comandos no terminal.

Reflexão Final

Apesar de ser uma implementação pequena, o planejamento prévio rigoroso deixou o processo de desenvolvimento muito tranquilo e estruturado, evitando horas de *debug* por falta de entendimento. Se o escopo e o tempo permitissem, eu implementaria uma função `main` completa e funcional do zero (além do arquivo de testes) e adicionaria funções utilitárias extras para melhorar o gerenciamento da estação. Também buscaria blindar a aplicação tratando situações fora do enunciado original. A principal lição de organização que levo e aplicarei de forma diferente na próxima vez é a documentação simultânea, preencher o relatório passo a passo durante a implementação, evitando deixar toda a escrita técnica para depois que o código já está finalizado.

Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	ok
Rodei os testes e evidencias.log foi gerado	ok
Tenho prints obrigatórios	ok
Testei pelo menos um caso limite e um caso inválido	ok
Preenchi a seção de uso de IA	ok
Revisei o relatório e removi frases genéricas / vazias	ok

Se eu tivesse mais 2 horas

- Implementação de uma `main` própria: Desenvolver uma função principal robusta para testar limites fora do escopo original, como o suporte a dois vagões concorrentes na mesma estação e o tratamento defensivo para chamadas repetidas de `station_on_board` por um mesmo passageiro.
- Robustez: impedir que `passageiros_embarcando` fique negativo se `station_on_board` for chamado a mais. *Verificação:* um teste que chama `on_board` a mais e confere o contador.
- Warnings do TSan: criar uma cópia do teste do professor com variáveis atômicas e `pthread_join`. *Verificação:* rodar o TSan e obter zero avisos.
- Organização: criar um alvo `make test` que roda os testes em loop. *Verificação:* `make clean && make test` termina sem erros.